Aula 36: Testes em Componentes React

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

Índice

- INTRODUÇÃO
- TESTE MANUAL VS AUTOMATIZADO
- TESTES EM REACT
- JEST
- REACT TESTING LIBRARY (RTL)
- TESTES UNITÁRIOS
- TESTES DE INTEGRAÇÃO
- TESTES DE UNIDADE VS INTEGRAÇÃO
- BOAS PRÁTICAS E ERROS COMUNS
- EXERCÍCIOS

Introdução

- O teste de software é o processo de avaliar e verificar se um produto de software ou aplicação funciona corretamente, com segurança e eficiência, de acordo com seus requisitos específicos
- No mundo interconectado de hoje, as consequências dos defeitos de software são mais graves do que nunca
- Atrasos na entrega ou falhas podem prejudicar a reputação de uma marca
- Em casos extremos, um bug ou defeito pode degradar sistemas interconectados ou causar falhas graves

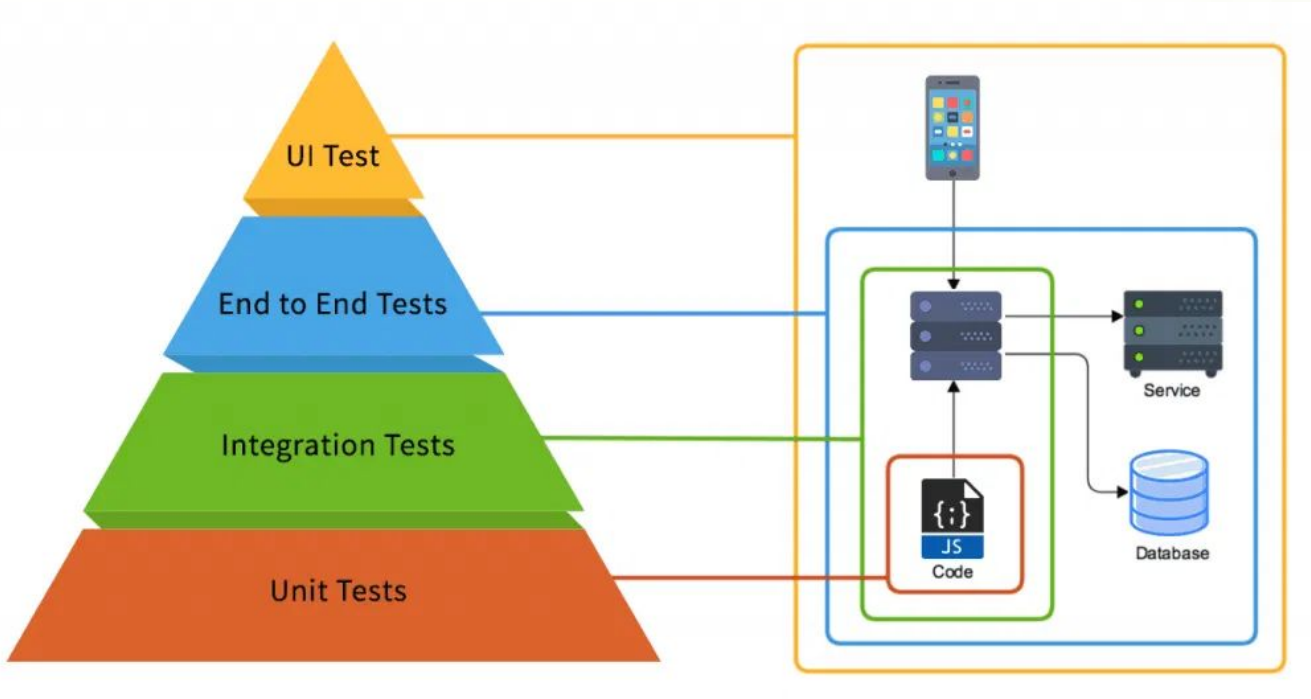
Introdução

- O teste em si envolve custos
- Porém empresas podem economizar bastante em desenvolvimento e suporte ao implementar técnicas de teste eficazes e processos de QA
- O teste de software antecipado identifica problemas antes que um produto seja lançado no mercado
- Quanto mais cedo as equipes de desenvolvimento recebem o feedback dos testes, mais cedo podem resolver problemas críticos

Introdução

NÍVEIS DE TESTE DE SOFTWARE

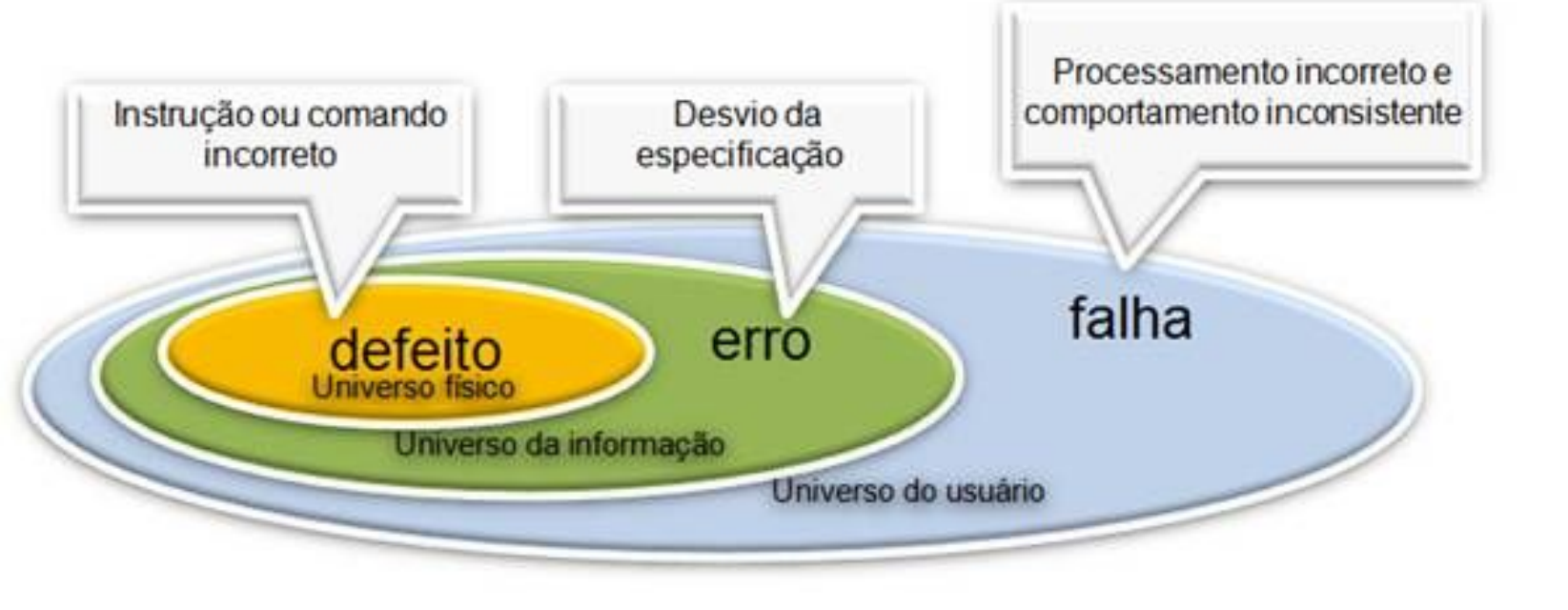
- De modo geral, o teste de software ocorre em quatro níveis:
  - Teste de aceitação
  - Teste de sistema
  - Testes de integração
  - Teste unitário



Introdução

TESTE DE ACEITAÇÃO

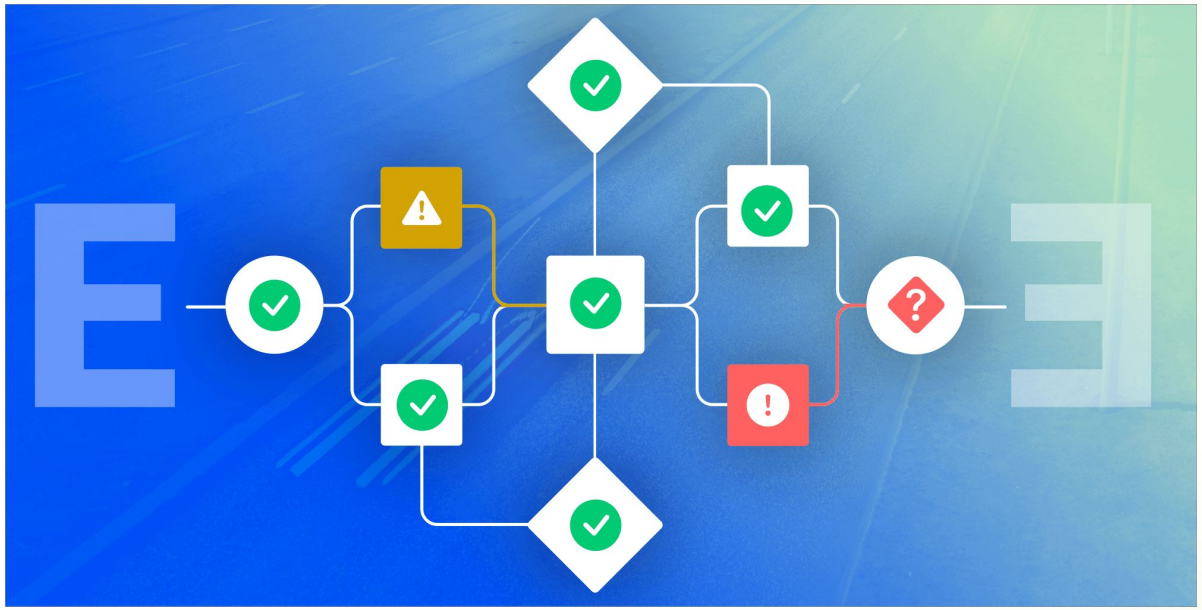
- O teste de aceitação verifica se todo o sistema funciona conforme o esperado



Introdução

TESTE DE SISTEMA

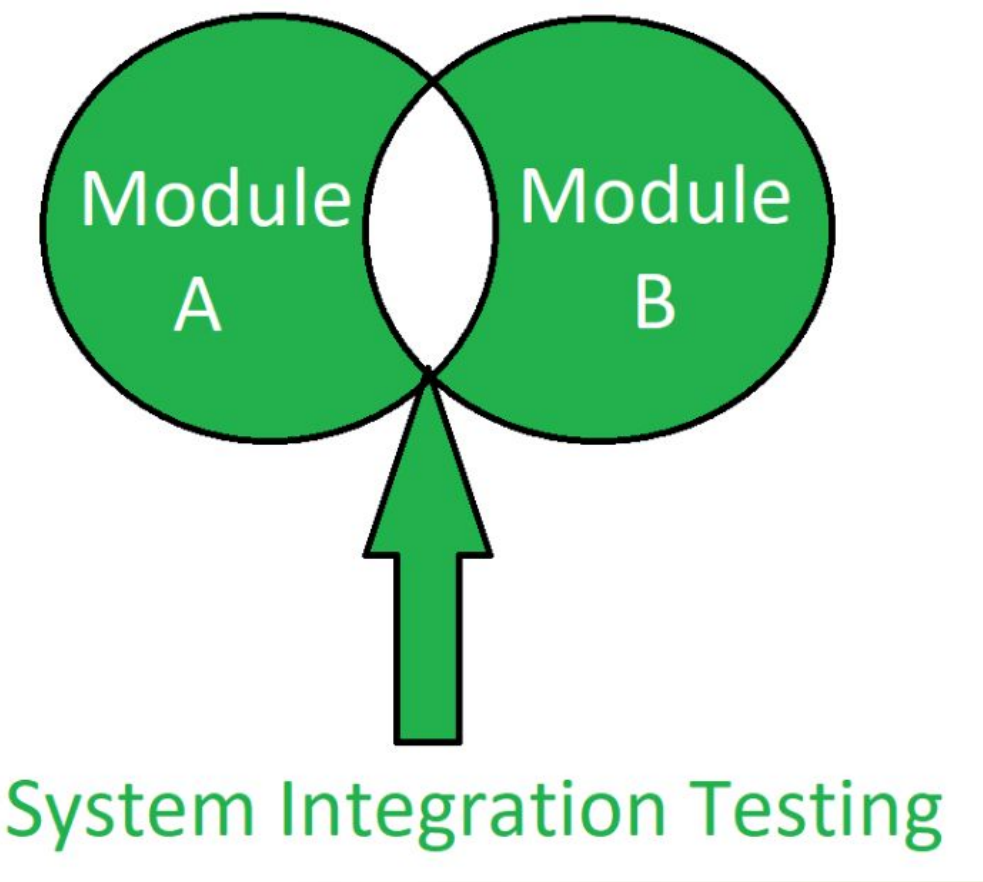
- O teste de sistema abrange o desempenho de ponta a ponta de todo o sistema
- Essa fase inclui aspectos de teste funcional, não funcional, de interface, de estresse e de recuperação



Introdução

TESTE DE INTEGRAÇÃO

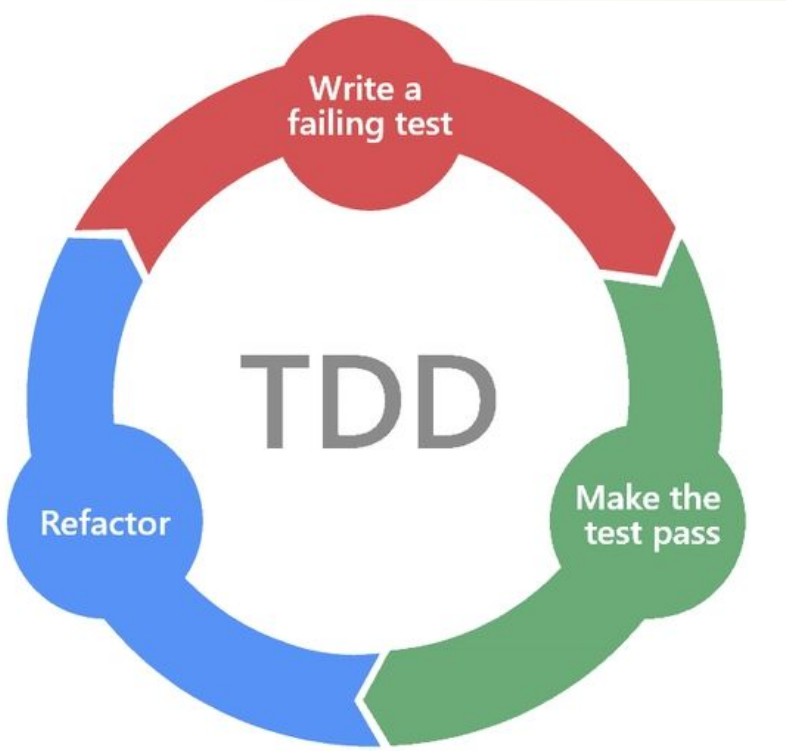
- O teste de integração garante que componentes ou funções de software funcionem em conjunto de forma eficaz



Introdução

TESTE DE UNIDADE

- O teste de unidade valida se cada unidade de software é executada conforme esperado
- Uma unidade é o menor componente testável de uma aplicação



Enquanto os testes de unidade focam em isolar e validar a lógica de componentes simples ou funções puras, os testes de integração verificam a colaboração entre diferentes partes do sistema.

Teste Manual vs Automatizado

TESTE MANUAL

- É o processo no qual os testadores executam casos de teste manualmente, sem o auxílio de ferramentas de automação
- Os testadores realizam ações como clicar em botões, inserir texto e verificar resultados, simulando como um usuário final interagiria com o software
- É normalmente usado para testes exploratórios, de usabilidade e quando a aplicação é pequena o suficiente para que não seja necessário automatizá-la

Teste Manual vs Automatizado

TESTES AUTOMATIZADOS

- Usa scripts e ferramentas para executar testes em software automaticamente
- Abordagem fundamental e benéfica para tarefas de teste repetitivas e para sistemas maiores em que é necessário executar os mesmos testes várias vezes
- Garante que o software possa ser testado mais rapidamente e de forma consistente
- Reduz erros humanos e melhora a eficiência dos testes ao longo do tempo

Teste Manual vs Automatizado

TESTES AUTOMATIZADOS

- A adoção de testes automatizados funciona como uma rede de segurança estratégica
- Eles permitem que novas funcionalidades sejam integradas sem comprometer o comportamento de recursos já existentes

Teste Manual vs Automatizado

TESTES AUTOMATIZADOS

- AAA Pattern
  - Arrange (preparar)
  - Act (executar)
  - Assert (verificar)

Teste Manual vs Automatizado

TESTES AUTOMATIZADOS

- O que é um teste confiável?
  - Determinístico
  - Rápido
  - Independente

Testes em React

JEST E RTL

- O desenvolvimento moderno em React utiliza o Jest como o motor principal para execução de testes, gerenciamento de ambiente e geração de relatórios de cobertura
- Complementarmente, a React Testing Library (RTL) oferece utilitários que priorizam o teste do comportamento do usuário em detrimento dos detalhes de implementação interna.

JEST

O QUE É

- Framework de testes em JavaScript
- Criado pelo Facebook
- Integrado ao Create React App
- Exemplo Conceitual
  - Testar funções puras
  - Testes síncronos e assíncronos

JEST

PRINCIPAIS CONCEITOS

- describe
- test / it
- expect
- Matchers (toBe, toEqual, toContain, etc.)

JEST

- A Estrutura do Jest Organiza e Valida o Comportamento do Código
- Blocos de Organização
  - A organização dos testes no Jest é feita através de blocos semânticos
    - describe: agrupa cenários relacionados
    - test: define uma validação específica

JEST

EXPECT E MATCHERS

- Como dizemos ao Jest o que esperamos que aconteça.

Matcher	Uso
.toBe(value)	Igualdade exata (primitivos).
.toEqual(obj)	Igualdade profunda (objetos e arrays).
.toBeTruthy() / .toBeFalsy()	Valida valores booleanos ou equivalentes.
.toContain(item)	Verifica se um item está num array.
.toThrow()	Verifica se uma função dispara um erro.

JEST

CICLO DE VIDA

- Objetivo:

  Preparar o ambiente antes de cada teste e limpar depois

- O ciclo de vida dos testes é gerenciado por funções como beforeEach
- Garante que cada execução ocorra em um ambiente limpo e isolado
- Evita interferências entre os testes

JEST

CICLO DE VIDA

- Os 4 pilares:

1. beforeAll: Executa uma vez antes de todos os testes do bloco.
2. beforeEach: Executa antes de cada teste (ideal para renderizar componentes).
3. afterEach: Limpeza (resetar mocks, limpar o DOM).
4. afterAll: Executa uma vez ao final de tudo.

JEST

ORGANIZAÇÃO E ESCOPO

- describe('Nome do Grupo', () => { ... }): Agrupa testes relacionados (ex: "Testes do componente de Login")
- test() ou it(): Define o teste individual
- Dica: it é usado para ler como uma frase: "it should render..."

JEST

ORGANIZAÇÃO E ESCOPO

```
describe('Cálculos Matemáticos', () => {
  test('soma de dois números positivos', () => {
    expect(1 + 2).toBe(3);
  });
});
```

JEST

DICAS DE PRODUTIVIDADE

- test.only(...): Executa apenas aquele teste específico (útil quando você tem 100 testes e só um está falhando)
- test.skip(...): Pula um teste temporariamente
- Modo Watch: jest --watch (Executa testes automaticamente ao salvar arquivos)

JEST

EXEMPLO

```
const user = { name: 'Ana' };
expect(user).toBe({ name: 'Ana' });    // FALHA (referência de memória diferente)
expect(user).toEqual({ name: 'Ana' }); // PASSA (conteúdo igual)
```

JEST

EXEMPLO PRÁTICO

- Escreva um teste para uma função formatarMoeda(valor) que deve transformar 10 em “R$ 10,00”
- Use describe, test e expect

React Testing Library (RTL)

- Prioriza Consultas Baseadas em Acessibilidade e Uso Real
- A React Testing Library incentiva o uso de consultas que refletem a experiência do usuário
- Filosofia: "Testar componentes como o usuário os utiliza"
- Diferença entre RTL e Enzyme
  - Menos foco em implementação
  - Mais foco em comportamento

React Testing Library (RTL)

- **Conceito Chave**: O RTL não renderiza o componente no navegador real, mas em um ambiente simulado chamado jsdom.
- **A função render**: Ponto de partida.
  - Ela coloca o componente no DOM virtual para que possamos interagir com ele.
- **O objeto screen**: o "olho do usuário"
  - É através dele que buscamos elementos na tela

React Testing Library (RTL)

- Principais Funções
  - render
  - screen
  - fireEvent
  - userEvent

React Testing Library (RTL)

- Prioriza métodos que tornam os testes mais robustos e promovem a acessibilidade

1. getByRole (Prioridade 1): Busca pela função semântica (button, heading, textbox). É a melhor prática.
2. getByLabelText: Ótimo para formulários.
3. getByPlaceholderText: Quando não há label.
4. getByText: Busca pelo conteúdo textual visível.
5. getByTestId: O último recurso. Usado quando não há nada semântico para agarrar

React Testing Library (RTL)

- Prefixos das Queries (get vs query vs find)
  - get...: Retorna o elemento ou dispara erro se não encontrar. (Use para o que deve estar lá)
  - query...: Retorna o elemento ou null. (Use para testar se algo não está na tela)
  - find...: Retorna uma Promise. (Use para elementos assíncronos que demoram a aparecer)
- getBy…, queryBy…, findBy…

Testes Unitários

- Validam a Renderização e Eventos Básicos
- Os testes de unidade em React começam com a renderização do componente em um ambiente virtual controlado
- O objetivo é garantir que a interface responda conforme o esperado às ações básicas, como cliques em botões ou preenchimento de campos simples

Testes Unitários

O QUE TESTAR

- Renderização correta
- Props
- Estados
- Eventos

Testes Unitários

EXEMPLO DE COMPONENTE

- Botão
- Contador
- Input controlado

EXEMPLO DE TESTES

- Verificar texto na tela
- Simular clique
- Validar mudança de estado

Testes Unitários

- Através da ferramenta userEvent, é possível simular interações humanas realistas
- Diferente do fireEvent, o userEvent dispara todos os eventos de navegador necessários, como foco e mudança de estado, tornando o teste mais fiel ao uso real
- Embora exista o fireEvent, o user-event é superior porque simula interações completas
- ex: ao digitar, ele dispara os eventos de keydown, keypress e keyup

Testes Unitários

- A biblioteca user-event:

```
import userEvent from '@testing-library/user-event';

test('deve permitir digitar no campo de busca', async () => {
  render(<Search />);
  const input = screen.getByRole('textbox');

  await userEvent.type(input, 'React');

  expect(input.value).toBe('React');
});
```

Testes Unitários

- A biblioteca user-event:

```
import userEvent from '@testing-library/user-event';

test('deve permitir digitar no campo de busca', async () => {
  render(<Search />);
  const input = screen.getByRole('textbox');

  await userEvent.type(input, 'React');

  expect(input.value).toBe('React');
});
```

Interações com user-event são quase sempre assíncronas (await).

Testes de Integração

O QUE SÃO

- Vários componentes juntos
- Fluxo completo do usuário

EXEMPLOS

- Formulário com validação
- Lista + botão de adicionar
- Consumo de API (mock)

EXEMPLO DE FLUXO: CONTADOR

1. Renderizar o componente pai que contém o estado e os botões.
2. Localizar o botão de incremento e o texto que exibe o valor atual.
3. Simular múltiplos cliques no botão usando userEvent.
4. Verificar se a interface reflete o novo valor acumulado corretamente.

Testes de Integração

- Diferente dos testes isolados, os testes de integração focam no fluxo de trabalho completo
- Considera-se como múltiplos elementos cooperam para atingir um objetivo
- O teste deve observar como a interface do usuário reage às mudanças de estado após sucessivas interações
- Ele não se preocupa com a lógica interna de gerenciamento de dados

Testes de Integração

- Essa abordagem garante que a experiência final do usuário seja preservada
- Isso ocorre independentemente de como o código foi estruturado internamente ou hooks utilizados

EXEMPLO DE FLUXO: CONTADOR

1. Renderizar o componente pai que contém o estado e os botões.
2. Localizar o botão de incremento e o texto que exibe o valor atual.
3. Simular múltiplos cliques no botão usando userEvent.
4. Verificar se a interface reflete o novo valor acumulado corretamente.

Testes de Integração

- Assincronicidade e Mocks Garantem Testes Rápidos e Isolados
- Para manter a velocidade e a estabilidade dos testes, é fundamental evitar chamadas reais a APIs externas
- Utiliza-se técnicas de mocking para simular respostas de rede

```
/ Mockando o módulo axios
jest.mock('axios');

test('exibe dados da API', async () => {
  render(<UserList />);

  // Espera o elemento aparecer
  const user = await
screen.findByText(/joão/i);

  expect(user).toBeInTheDocument();
});
```

Testes de Integração

- O Jest permite criar funções espiãs e substituir módulos inteiros, como o Axios, por versões controladas que retornam dados instantaneamente
- Respostas simuladas:
  - Queries assíncronas como findByText permitem que o teste aguarde a atualização da interface antes de realizar as asserções finais

Testes de Unidade vs Integração

- Unidade:
  - Foco: Um único componente.
  - Ex: O botão muda de cor ao ser clicado?
- Integração:
  - Foco: A comunicação entre vários componentes.
  - Ex: Ao preencher o formulário e clicar em enviar, a mensagem de sucesso aparece e o input é limpo?

Boas Práticas e Erros Comuns

BOAS PRÁTICAS

- Priorizar getByRole
- Testar comportamento, não implementação
- Nomear testes claramente

ERROS COMUNS

- Testes frágeis
- Uso excessivo de querySelector
- Testar detalhes internos do componente

Exercício 1

- Criar um componente Counter com:
  - Valor inicial 0
  - Botão "Incrementar"
  - Exibição do valor atual
- Testes esperados:
  - Renderiza o valor inicial
  - Incrementa ao clicar no botão

Exercício 2

- Criar um componente Formulário com:
  - Campo de nome
  - Botão enviar
  - Mensagem de sucesso
- Testes esperados:
  - Campo renderiza corretamente
  - Usuário consegue digitar
  - Mensagem aparece após envio

Exercício: Gerenciador de Tarefas

- Ao longo deste curso, vamos desenvolver progressivamente uma aplicação React completa, aplicando os conceitos estudados em cada aula
- O projeto será construído de forma incremental: A cada novo conteúdo, novas funcionalidades deverão ser adicionadas
- Ao final do curso, você teremos:
  - Uma aplicação React funcional
  - Um projeto organizado, testado e otimizado
  - Um deploy público em ambiente de produção

Exercício: Gerenciador de Tarefas

- O sistema consiste em uma aplicação web SPA (Single Page Application) para gerenciamento de projetos e tarefas, permitindo que o usuário:
  - Visualize projetos cadastrados
  - Acesse tarefas associadas a cada projeto
  - Crie, edite, conclua e remova tarefas
  - Navegue entre diferentes páginas da aplicação
  - Tenha seus dados persistidos localmente

Exercício: Gerenciador de Tarefas

- A aplicação deverá conter, no mínimo, os seguintes componentes:
  - App
  - Header
  - ProjectList
  - TaskList
  - TaskItem
  - TaskForm

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Testes em Componentes React**
  - Configurar Jest/Testing Library.
  - Teste 1: Verificar se o título "Gerenciador de Tarefas" aparece na tela
  - Teste 2: Simular a digitação no input e o clique em "Adicionar", verificando se o item apareceu na lista.
- Funcionalidade: Testar se o botão adiciona o item na tela

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Testes em Componentes React**
- Conceitos implementados:
  - Criação de testes de unidade e integração
  - Testes de renderização e interação do usuário
  - Uso do React Testing Library
- Resultado Visual:
  - O terminal mostrando "PASS" em verde